

Agentic Theory: The Artificial Junky Neuron (AJN) and Its Integration into Multi-Layer Neural Architectures

Justo Tapiador García

Department of Artificial Intelligence and Neuroscience
Universidad de Alicante (UA)

April 24, 2026

Abstract

This paper extends the theoretical framework of the Artificial Junky Neuron (AJN)—an agent-like computational unit that develops a dependency on specific classes of input stimuli—to address its integration within multi-layer neural network architectures. We first consolidate the single-unit model with a fully formalized state tuple, explicit decay dynamics, and a rigorous comparison with classical reinforcement learning agents. We then define three compositional paradigms for deploying AJN units: (i) homogeneous AJN layers, in which all units share the same stimulus class and cooperate through inter-unit praxic coupling; (ii) heterogeneous AJN layers, in which units encode competing addiction profiles, giving rise to emergent specialization; and (iii) hybrid architectures that alternate classical feedforward layers with AJN layers, preserving gradient-based optimization while injecting intrinsic motivation at selected processing depths. We formalize the inter-unit coupling tensor, the layer-level consensus mechanism, and the conditions under which a population of AJNs achieves collective saturation or descends into synchronized chaos. We further characterize the network-level extinction cascade and propose an extinction-prevention regularization term. The resulting framework, which we call the *Agentic Neural Network* (ANN- Ψ), constitutes a new class of adaptive architecture in which motivation, exploration, and goal-directed behavior emerge from the microscopic dynamics of individual units rather than from a global reward signal.

Contents

1	Introduction	3
2	The Consolidated AJN Model	3
2.1	Tensorial Space of Stimuli	3
2.2	The Complete AJN State Tuple	3
2.3	Craving Dynamics	4
2.4	Threshold Dynamics	4

2.5	Policy Generator and Praxic Distribution	4
2.6	Complete Phase Diagram	5
2.7	Algorithmic Summary	5
3	Layer-Level Integration of AJN Units	5
3.1	Paradigm I: Homogeneous AJN Layer	6
3.2	Paradigm II: Heterogeneous AJN Layer	6
3.3	Paradigm III: Hybrid Agentic–Classical Layers	7
4	The Agentic Neural Network (ANN-Ψ)	7
4.1	Architecture Definition	7
4.2	Dual Optimization Objective	7
4.3	Extinction Cascade and Regularization	8
4.4	Complete Network Algorithm	8
5	Discussion	9
5.1	Relationship to Reinforcement Learning	9
5.2	Free Energy and the AJN	9
5.3	Emergent Specialization in Heterogeneous Layers	9
5.4	Limitations and Open Questions	9
6	Conclusion	10

1 Introduction

Classical artificial neural networks are organized as passive, feedforward (or recurrent) stacks of weighted summation units followed by non-linear activation functions. Learning is a macro-level process driven by a scalar loss signal backpropagated through the architecture. Individual units have no internal motivation; they merely compute.

The *Agentic Theory* challenges this paradigm by attributing to each computational unit an autonomous drive: the unit actively seeks to increase the intensity of a specific class of sensory input. In a previous communication [1], the **Artificial Junky Neuron (AJN)** was introduced as the atomic element of such a theory. Its dynamics encompass six phases—random exploration, reinforcement, saturation, withdrawal, frustration/chaos, and extinction—governed by an internal addiction cycle.

The natural and open question is: *what happens when many AJNs are organized into layers and stacked into a network?* This paper answers that question in three steps. In Section 2 we consolidate and extend the single-unit model, filling the formalization gaps left in the original work. In Section 3 we define three layer-level compositional paradigms and derive their collective dynamics. In Section 4 we present the full Agentic Neural Network (ANN- Ψ) architecture, its training procedure, and its relationship to existing deep learning and reinforcement learning frameworks.

2 The Consolidated AJN Model

2.1 Tensorial Space of Stimuli

Let the environment provide a stimulus tensor at discrete time step t :

$$\mathcal{S}_t \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_k}.$$

The *Stimulus Intensity Function* $I : \mathbb{R}^{d_1 \times \dots \times d_k} \rightarrow [0, 1]$ maps the tensor to a normalized scalar:

$$\alpha_t = I(\mathcal{S}_t), \quad \alpha_t \in [0, 1].$$

Different AJN units may instantiate different functions $I^{(j)}$, encoding distinct “objects of addiction.” A natural choice is a learned inner product with a prototype template $\mathbf{W}_I$:

$$I(\mathcal{S}_t) = \sigma(\langle \mathbf{W}_I, \mathcal{S}_t \rangle_F),$$

where σ is the logistic function and $\langle \cdot, \cdot \rangle_F$ the Frobenius inner product.

2.2 The Complete AJN State Tuple

We now extend the original three-element tuple to a five-element specification:

$$\text{AJN} \triangleq (\mathcal{M}, \theta, \Omega, \delta, \tau),$$

where:

- $\mathcal{M}(t) \in [0, 1]$ is the *craving level* (addiction magnitude) at time t , with dynamics defined in Section 2.3.

- $\theta(t) \in [0, 1]$ is the *activation threshold*, which rises upon saturation and decays during withdrawal; its dynamics are given in Section 2.4.
- $\Omega : \mathcal{H}_t \times \mathcal{M}(t) \rightarrow \mathcal{P}_t$ is the *policy generator*, mapping the history $\mathcal{H}_t$ and the current craving level to a praxis tensor.
- $\delta > 0$ is the *metabolic decay rate* governing the speed of withdrawal after saturation.
- $\tau > 0$ is the *extinction horizon*, the number of consecutive failure steps before addiction extinction is triggered.

2.3 Craving Dynamics

The craving level $\mathcal{M}(t)$ evolves according to a stimulus-driven integro-differential equation:

$$\mathcal{M}(t+1) = \beta_M \cdot \mathcal{M}(t) + (1 - \beta_M) \cdot \alpha_t, \quad (1)$$

where $\beta_M \in (0, 1)$ is an exponential smoothing factor. Equation (1) captures tolerance: repeated intense stimulation accelerates $\mathcal{M}(t) \rightarrow 1$, raising the bar for future satisfaction.

2.4 Threshold Dynamics

The activation threshold $\theta(t)$ is governed by two opposing forces:

$$\theta(t+1) = \begin{cases} \min(1, \theta(t) + \lambda_{\uparrow} \cdot \alpha_t) & \text{if } \alpha_t > \theta_{\text{sat}} \quad (\text{satisfaction}) \\ \max(0, \theta(t) - \delta) & \text{otherwise} \quad (\text{withdrawal decay}) \end{cases} \quad (2)$$

where $\lambda_{\uparrow} > 0$ is the saturation ascent rate and $\delta > 0$ is the withdrawal decay rate introduced in Section 2.2. As θ descends below $\mathcal{M}(t)$, the craving becomes actionable and the AJN enters the praxic cycle.

2.5 Policy Generator and Praxic Distribution

The policy generator Ω maintains a Gaussian distribution over the praxis tensor space:

$$\mathcal{P}_t \sim \mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t),$$

where $\boldsymbol{\mu}_t \in \mathbb{R}^m$ is the mean praxis and $\Sigma_t \in \mathbb{R}^{m \times m}$ (diagonal in practice) is the covariance. These parameters update as follows:

On success ($\Delta\alpha_t > 0$):

$$\boldsymbol{\mu}_{t+1} = \boldsymbol{\mu}_t + \eta \cdot \Delta\alpha_t \cdot \nabla_{\mathcal{P}} \alpha_t, \quad (3)$$

$$\Sigma_{t+1} = \Sigma_t \cdot \exp(-\lambda_{\Sigma} \cdot \Delta\alpha_t), \quad (4)$$

where $\eta > 0$ is the praxic learning rate and $\lambda_{\Sigma} > 0$ controls entropy reduction.

On failure ($\Delta\alpha_t \leq 0$), **frustration intensification**:

$$\boldsymbol{\mu}_{t+1} = \boldsymbol{\mu}_t, \quad (5)$$

$$\Sigma_{t+1} = \Sigma_t \cdot \exp(+\gamma \cdot |\Delta\alpha_t|), \quad (6)$$

where $\gamma > 0$ governs the rate of chaotic expansion.

Extinction ($n_{\text{fail}} > \tau$):

$$\boldsymbol{\mu}_{t+1} \leftarrow \mathbf{0}, \quad \Sigma_{t+1} \leftarrow \Sigma_{\text{max}} \cdot \mathbf{I}, \quad \mathcal{M}(t+1) \leftarrow 0. \quad (7)$$

2.6 Complete Phase Diagram

Figure 1 summarizes the six-phase life-cycle of an AJN as a finite state machine.

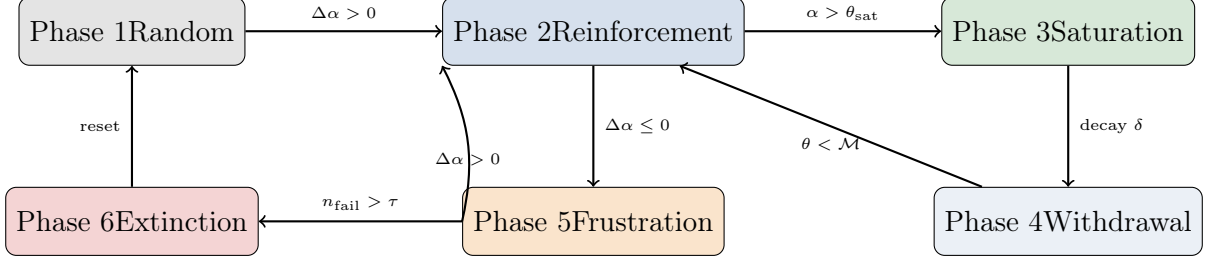


Figure 1: Finite state machine representation of the AJN life-cycle. Transitions are labelled by the triggering conditions.

2.7 Algorithmic Summary

Algorithm 1 Extended AJN Processing Cycle (single unit)

- 1: **Input:** Stimulus $\mathcal{S}_t$; state $(\mathcal{M}(t), \theta(t), \boldsymbol{\mu}_t, \Sigma_t, n_{\text{fail}})$
 - 2: Compute intensity: $\alpha_t \leftarrow I(\mathcal{S}_t)$
 - 3: Update craving: $\mathcal{M}(t+1) \leftarrow \beta_M \mathcal{M}(t) + (1-\beta_M)\alpha_t$
 - 4: **if** $\alpha_t > \theta_{\text{sat}}$ **then**
 - 5: **[Saturation]** Output $\mathcal{P}_t \leftarrow \mathbf{0}$
 - 6: $\theta(t+1) \leftarrow \min(1, \theta(t) + \lambda_{\uparrow} \alpha_t)$; $n_{\text{fail}} \leftarrow 0$
 - 7: **else**
 - 8: $\theta(t+1) \leftarrow \max(0, \theta(t) - \delta)$ *// Withdrawal decay*
 - 9: Sample: $\mathcal{P}_t \sim \mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t)$
 - 10: Compute $\Delta\alpha_t \leftarrow \alpha_{t+1} - \alpha_t$ *// Observe response*
 - 11: **if** $\Delta\alpha_t > 0$ **then**
 - 12: Update $\boldsymbol{\mu}_{t+1}, \Sigma_{t+1}$ via Eqs. (3)–(4); $n_{\text{fail}} \leftarrow 0$
 - 13: **else**
 - 14: Expand Σ_{t+1} via Eq. (6); $n_{\text{fail}} \leftarrow n_{\text{fail}} + 1$
 - 15: **if** $n_{\text{fail}} > \tau$ **then**
 - 16: **[Extinction]** Reset via Eq. (7)
 - 17: **end if**
 - 18: **end if**
 - 19: **end if**
 - 20: **Output:** Praxis tensor $\mathcal{P}_t$ to transducers
-

3 Layer-Level Integration of AJN Units

We now address the central contribution of this paper: how AJN units compose into *layers* and how those layers interact. We identify three integration paradigms.

3.1 Paradigm I: Homogeneous AJN Layer

Let a layer ℓ contain N AJN units, all sharing the same stimulus class (i.e., $I^{(j)} = I$ for all $j = 1, \dots, N$). Each unit generates its own praxis $\mathcal{P}_t^{(j)}$. We define the *layer praxis* as the aggregate:

$$\overline{\mathcal{P}}_t^{(\ell)} = \frac{1}{N} \sum_{j=1}^N \mathcal{P}_t^{(j)}. \quad (8)$$

The layer praxis is passed to the transducer set $\mathcal{T}^{(\ell)}$ to produce the next-layer stimulus:

$$\mathcal{S}_{t+1}^{(\ell+1)} = \mathcal{E}^{(\ell)} \left(\mathcal{T}^{(\ell)} \left(\overline{\mathcal{P}}_t^{(\ell)} \right), \mathcal{S}_t^{(\ell)} \right). \quad (9)$$

Inter-unit coupling. To prevent the layer from collapsing into all units simultaneously entering Phase 3 (collective saturation lock), we introduce a *coupling inhibition tensor*:

$$\hat{\alpha}_t^{(j)} = \alpha_t - \kappa \cdot \frac{1}{N-1} \sum_{k \neq j} \mathcal{M}_t^{(k)}, \quad (10)$$

where $\kappa \geq 0$ is the coupling coefficient. When neighbouring units are highly addicted ($\mathcal{M}_t^{(k)} \approx 1$), the effective stimulus intensity available to unit j is reduced, creating competitive pressure and asynchronous saturation cycles. This lateral inhibition prevents degenerate consensus and maintains exploration diversity within the layer.

Collective saturation. The layer enters collective saturation when:

$$\frac{1}{N} \sum_{j=1}^N \mathbf{1} \left[\alpha_t > \theta_{\text{sat}}^{(j)} \right] > \rho_{\text{sat}}, \quad (11)$$

for a threshold fraction $\rho_{\text{sat}} \in (0, 1]$. At this point the layer output is suppressed and the downstream layer receives no praxic input.

3.2 Paradigm II: Heterogeneous AJN Layer

Let units $j = 1, \dots, N$ instantiate K distinct stimulus classes, with n_k units assigned to class k ($\sum_k n_k = N$). Units in the same class cooperate (share a common $\boldsymbol{\mu}$ update), while units in different classes compete for the transducer bandwidth.

The layer output is now a *class-resolved praxis map*:

$$\mathcal{P}_t^{(k)} = \frac{1}{n_k} \sum_{j: I^{(j)}=I_k} \mathcal{P}_t^{(j)}, \quad k = 1, \dots, K. \quad (12)$$

The winning class k^* is selected by a soft-max competition over addiction levels:

$$k^* = \arg \max_k \bar{\mathcal{M}}_t^{(k)}, \quad \bar{\mathcal{M}}_t^{(k)} = \frac{1}{n_k} \sum_{j: I^{(j)}=I_k} \mathcal{M}_t^{(j)}. \quad (13)$$

Only the winning class issues praxes to the transducers, while the remaining classes continue to evolve internally. This mechanism gives rise to spontaneous *task specialization* without explicit supervision.

3.3 Paradigm III: Hybrid Agentic–Classical Layers

The most pragmatic architecture for integration with existing deep learning pipelines is a hybrid stack, alternating classical feedforward (or convolutional) layers with AJN layers:

$$\text{Input} \rightarrow \underbrace{\text{FC/Conv}}_{\text{feature extraction}} \rightarrow \underbrace{\text{AJN Layer}}_{\text{intrinsic motivation}} \rightarrow \underbrace{\text{FC/Conv}}_{\text{abstraction}} \rightarrow \cdots \rightarrow \text{Output}.$$

In this paradigm, the AJN layer receives the feature map produced by the preceding classical layer as its stimulus tensor $\mathcal{S}_t$, and its aggregate praxis $\bar{\mathcal{P}}_t$ is concatenated (or added residually) to the feature map before passing it to the next classical layer:

$$\tilde{\mathcal{S}}_t^{(\ell+1)} = f^{(\ell)}(\mathcal{S}_t^{(\ell)}) \oplus \bar{\mathcal{P}}_t^{(\ell)}, \quad (14)$$

where $f^{(\ell)}$ is the classical layer transformation and $\oplus$ denotes channel-wise concatenation. The AJN praxis thus acts as a *context modulation signal*, injecting goal-directed bias into the feature stream without disrupting the gradient flow through the classical layers.

4 The Agentic Neural Network (ANN- Ψ)

4.1 Architecture Definition

An Agentic Neural Network of depth L is defined as a sequence of layers

$$\text{ANN-}\Psi = \{\mathcal{L}^{(1)}, \mathcal{L}^{(2)}, \dots, \mathcal{L}^{(L)}\},$$

where each $\mathcal{L}^{(\ell)}$ is one of: a classical feedforward layer, a homogeneous AJN layer (Paradigm I), a heterogeneous AJN layer (Paradigm II), or a hybrid layer (Paradigm III). The overall network dynamics at time t are:

$$\mathcal{S}_t^{(\ell+1)} = \mathcal{L}^{(\ell)}(\mathcal{S}_t^{(\ell)}, \mathcal{H}_t^{(\ell)}), \quad \ell = 1, \dots, L, \quad (15)$$

where $\mathcal{H}_t^{(\ell)}$ is the layer-local history (aggregated unit states).

4.2 Dual Optimization Objective

Training ANN- Ψ involves two objectives that operate at different timescales:

Classical objective (macro-level). The network is optimized end-to-end via gradient descent on a supervised or self-supervised loss $\mathcal{L}_{\text{task}}(\hat{y}, y)$, backpropagated through the classical and hybrid layers. The AJN layers are treated as frozen “noise injectors” during this pass, contributing through the hybrid concatenation in Eq. (14).

Agentic objective (micro-level). Each AJN unit independently optimizes its own stimulus-seeking objective:

$$\mathcal{J}^{(j)} = \mathbb{E}_t[\alpha_t^{(j)}] - \lambda_{\text{ext}} \cdot \mathbf{1}[n_{\text{fail}}^{(j)} > \tau], \quad (16)$$

where the second term penalizes extinction events. The policy gradient update for the praxis distribution parameters $(\boldsymbol{\mu}^{(j)}, \Sigma^{(j)})$ follows directly from Section 2.5.

4.3 Extinction Cascade and Regularization

A critical failure mode of ANN- Ψ is the *extinction cascade*: if a lower layer’s AJN units enter Phase 6 (extinction) simultaneously, the praxic context signal vanishes, starving upper layers of modulatory input. We define the *cascade risk* at layer ℓ as:

$$\rho_{\text{ext}}^{(\ell)} = \frac{1}{N_\ell} \sum_{j=1}^{N_\ell} \mathbf{1} \left[n_{\text{fail}}^{(j,\ell)} > \frac{\tau}{2} \right], \quad (17)$$

measuring the fraction of units approaching extinction. To prevent cascades, we add a regularization term to the macro-level loss:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{cas}} \sum_{\ell=1}^{L_{\text{AJN}}} \rho_{\text{ext}}^{(\ell)}, \quad (18)$$

where L_{AJN} is the number of AJN layers and $\lambda_{\text{cas}} > 0$ is the cascade regularization coefficient. Minimizing $\mathcal{L}_{\text{reg}}$ encourages the network to present sufficiently rich stimuli to each AJN layer, preventing units from starving.

The combined training objective is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}}(\hat{y}, y) + \lambda_{\text{cas}} \sum_{\ell} \rho_{\text{ext}}^{(\ell)}. \quad (19)$$

4.4 Complete Network Algorithm

Algorithm 2 ANN- Ψ Forward Pass and Update

- 1: **Input:** External stimulus $\mathcal{S}_t^{(0)}$; network $\{\mathcal{L}^{(\ell)}\}_{\ell=1}^L$; target y
 - 2: **for** $\ell = 1$ to L **do**
 - 3: **if** $\mathcal{L}^{(\ell)}$ is a classical layer **then**
 - 4: $\mathcal{S}_t^{(\ell)} \leftarrow f^{(\ell)}(\mathcal{S}_t^{(\ell-1)})$
 - 5: **else if** $\mathcal{L}^{(\ell)}$ is a homogeneous AJN layer **then**
 - 6: **for** each unit j in layer ℓ **do**
 - 7: Run Algorithm 1 to get $\mathcal{P}_t^{(j)}$
 - 8: **end for**
 - 9: Compute $\bar{\mathcal{P}}_t^{(\ell)}$ via Eq. (8)
 - 10: $\mathcal{S}_t^{(\ell)} \leftarrow \mathcal{E}^{(\ell)}(\mathcal{T}^{(\ell)}(\bar{\mathcal{P}}_t^{(\ell)}), \mathcal{S}_t^{(\ell-1)})$
 - 11: **else if** $\mathcal{L}^{(\ell)}$ is a hybrid layer **then**
 - 12: $\mathcal{S}_t^{(\ell)} \leftarrow f^{(\ell)}(\mathcal{S}_t^{(\ell-1)}) \oplus \bar{\mathcal{P}}_t^{(\ell)}$ // Eq. (14)
 - 13: **end if**
 - 14: **end for**
 - 15: Compute $\hat{y} = \mathcal{L}^{(L)}(\mathcal{S}_t^{(L-1)})$
 - 16: Compute $\mathcal{L}_{\text{total}}$ via Eq. (19)
 - 17: **Macro-update:** $\nabla_{\mathbf{w}} \mathcal{L}_{\text{total}}$ (backprop through classical/hybrid layers)
 - 18: **Micro-update:** For each AJN unit, update $(\mu^{(j)}, \Sigma^{(j)})$ via Eqs. (3)–(6)
-

5 Discussion

5.1 Relationship to Reinforcement Learning

The AJN shares structural elements with standard reinforcement learning (RL) agents [2]: both maintain a policy and update it from environmental feedback. However, there are fundamental distinctions:

Table 1: Comparison of AJN with classical RL agents.

Property	RL Agent (Q-Learning/Actor-Critic)	AJN
Reward source	External scalar signal	Intrinsic stimulus intensity
Policy level	Global (whole network)	Microscopic (single unit)
Exploration	ε -greedy / entropy bonus	Phase-driven chaos
State	Markov state of environment	Internal craving + threshold
Extinction	Not modelled	Explicit Phase 6
Architecture	Separate agent + environment	Unit IS the agent

The closest existing construct to the AJN is the *intrinsic motivation* module in curiosity-driven RL [3], where an exploration bonus is derived from prediction error. The AJN differs in that: (a) the drive operates at the neuron level, not the agent level; (b) the extinction mechanism has no counterpart in curiosity models; and (c) the praxis tensor directly actuates transducers, closing the sensorimotor loop at the unit level.

5.2 Free Energy and the AJN

The AJN dynamics can be interpreted within Karl Friston’s Free Energy Principle [4]: the craving level $\mathcal{M}(t)$ represents a prior belief about the expected stimulus intensity, while the praxis sequences minimize the surprise (free energy) of observing a low stimulus. Saturation corresponds to a collapse of the variational free energy to zero; extinction corresponds to the abandonment of a generative model that has become unserviceable.

5.3 Emergent Specialization in Heterogeneous Layers

A key prediction of Paradigm II (Section ??) is that heterogeneous AJN layers spontaneously partition the input space: units addicted to class k attend to the features of class k , ignoring others. This mirrors the phenomenon of *grandmother cells* and *feature detectors* observed in biological cortex, and suggests that AJN layers could provide a gradient-free route to structured feature extraction.

5.4 Limitations and Open Questions

1. **Scalability.** The per-unit state maintenance grows linearly with the number of AJN units. Efficient implementations will require batched state tensors and GPU-parallel state machines.
2. **Gradient propagation.** The praxis policy update (Gaussian parameter update) is not differentiable with respect to the macro-level loss. Future work should explore reparameterized gradients or implicit differentiation through the policy.

3. **Stimulus class design.** The choice of $I^{(j)}$ for each unit is currently manual or random. Learning $I^{(j)}$ end-to-end would be a significant extension.
4. **Temporal credit assignment.** The AJN evaluates praxes based on the immediately subsequent stimulus. Multi-step temporal credit assignment (analogous to TD(λ)) remains an open problem for agentic layers.

6 Conclusion

We have extended the Artificial Junky Neuron framework from a single-unit model to a principled multi-layer architecture. The five-element AJN tuple, the explicit craving and threshold dynamics, and the three integration paradigms (homogeneous, heterogeneous, hybrid) together constitute the foundation of the Agentic Neural Network (ANN- Ψ).

The central insight is that *motivation need not be global*. A network composed of thousands of micro-agents, each pursuing its own addictive stimulus, can exhibit collective goal-directed behavior, spontaneous specialization, and resilience through independent extinction–reactivation cycles—without any single agent or loss function orchestrating the whole. This bottom-up emergence of purposive behavior represents a genuinely new organizing principle for artificial intelligence systems.

Acknowledgments

The author thanks the theoretical framework provided by the study of agentic systems, neural feedback loops, and the Free Energy Principle. This work builds directly upon the foundational AJN definition in [1].

References

- [1] Tapiador García, J. (2024). Agentic Theory: Definition of the Artificial Junky Neuron (AJN). *Preprint, Universidad de Alicante*.
- [2] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [3] Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiosity-driven Exploration by Self-supervised Prediction. *Proceedings of ICML 2017*, 2778–2787.
- [4] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127–138.
- [5] Hebb, D. O. (1949). *The Organization of Behavior*. Wiley & Sons.
- [6] Skinner, B. F. (1938). *The Behavior of Organisms*. Copley Publishing Group.
- [7] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*.
- [8] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.

- [9] Vaswani, A., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- [10] Silver, D., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489.